2026-10-18 20:31:13 You

🟡 Level 5 — Templates

52. **Jinja2**
53. **Variables inside templates**
54. **Conditions inside templates**
55. **Loops inside templates**
56. **Template files (`.j2`)**
57. **Dynamic configuration generation**

ChatGPT gpt-6

Ansible Level 5 — Templates

Goal: Learn how Ansible generates configuration files dynamically using Jinja2, variables, conditions, and loops.

Core idea: Instead of manually maintaining different configuration files for different servers, create one template and let Ansible generate the correct file for each server.

52. Jinja2

Purpose: Jinja2 is a templating engine that allows Ansible to generate dynamic text using variables, conditions, loops, and expressions.

- Ansible uses Jinja2 primarily with the `template` module.
- Variables use `{{ }}`.
- Conditions and loops use `{% %}`.
- Comments use `{# #}`.

Example

`templates/app.conf.j2`

```jinja2
Application: {{ app_name }}
Port: {{ app_port }}
```

Playbook:

```yaml
- name: Generate configuration
  hosts: web
  vars:
    app_name: myapp
    app_port: 8080

  tasks:
    - name: Generate config file
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /tmp/app.conf
```

Generated `/tmp/app.conf`:

```
Application: myapp
Port: 8080
```

Remember:

Syntax	Purpose
`{{ variable }}`	Print a value
`{% ... %}`	Execute template logic
`{# ... #}`	Add a template comment

---

53. Variables Inside Templates

Purpose: Insert dynamic values into a configuration file.

Variables can come from playbooks, inventory files, host variables, group variables, or other Ansible variable sources.

Example

Template: `templates/nginx.conf.j2`

```nginx
server {
    listen {{ nginx_port }};
    server_name {{ domain_name }};
}
```

Playbook:

```yaml
- name: Configure Nginx
  hosts: web
  vars:
    nginx_port: 8080
    domain_name: example.com

  tasks:
    - name: Generate Nginx config
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /tmp/nginx.conf
```

Generated output:

```nginx
server {
    listen 8080;
    server_name example.com;
}
```

Practical example — Different values per server

Inventory:

```ini
[web]
web01 ansible_host=192.168.1.10 nginx_port=8080
web02 ansible_host=192.168.1.11 nginx_port=9090
```

Template:

```nginx
server {
    listen {{ nginx_port }};
}
```

Each server gets its own port in the generated file.

Remember: The same template can produce different configurations depending on the variables available to each host.

---

54. Conditions Inside Templates

Purpose: Generate different configuration content depending on a condition.

- Syntax: `{% if %}`, `{% elif %}`, `{% else %}`, `{% endif %}`.

Example

Template: `templates/app.conf.j2`

```jinja2
{% if environment == "production" %}
debug: false
log_level: ERROR
{% else %}
debug: true
log_level: DEBUG
{% endif %}
```

Playbook variables:

```yaml
vars:
  environment: production
```

Generated output:

```yaml
debug: false
log_level: ERROR
```

If `environment` is `development`, the output becomes:

```yaml
debug: true
log_level: DEBUG
```

Production use cases

- Enable debugging only in development.
- Set different log levels for staging and production.
- Include TLS settings only when TLS is enabled.

Remember: A template condition controls what gets written into the file. It is different from an Ansible `when` condition, which controls whether a task runs.

Example of a task condition:

```yaml
- name: Install Nginx only in production
  ansible.builtin.apt:
    name: nginx
    state: present
  when: environment == "production"
```

---

55. Loops Inside Templates

Purpose: Generate repeated configuration entries from a list.

- Syntax: `{% for item in list %}` and `{% endfor %}`.

Example — Generate multiple application ports

Playbook:

```yaml
vars:
  app_ports:
    - 3000
    - 3001
    - 3002
```

Template:

```jinja2
{% for port in app_ports %}
- "{{ port }}"
{% endfor %}
```

Generated output:

```yaml
- "3000"
- "3001"
- "3002"
```

Practical example — Generate Nginx upstream servers

Variables:

```yaml
vars:
  backend_servers:
    - "10.0.1.10:3000"
    - "10.0.1.11:3000"
    - "10.0.1.12:3000"
```

Template: `templates/upstream.conf.j2`

```nginx
```

```
upstream backend {
{% for server in backend_servers %}
    server {{ server }};
{% endfor %}
}
```

Generated output:

```
</> Nginx

upstream backend {
    server 10.0.1.10:3000;
    server 10.0.1.11:3000;
    server 10.0.1.12:3000;
}
```

Production use cases

- Generate backend server lists.
- Generate firewall rules.
- Generate application configuration entries.
- Generate monitoring targets.

Remember: An Ansible task loop repeats a task. A Jinja2 loop repeats content inside a generated file.

---

56. Template Files (`.j2`)

Purpose: Store reusable configuration templates separately from your playbook.

- `.j2` is a common naming convention for Jinja2 template files.
- The `template` module reads the source template, renders variables and logic, then writes the generated file to the target server.
- The remote destination usually has the actual configuration filename, without `.j2`.

Recommended project structure

```
</> Plain text

ansible-project/
├── inventory.ini
├── deploy.yml
└── templates/
    └── app.conf.j2
```

Template: `templates/app.conf.j2`

```jinja2
APP_NAME={{ app_name }}
APP_PORT={{ app_port }}
```

Playbook: `deploy.yml`

```yaml
- name: Deploy application configuration
  hosts: web
  vars:
    app_name: payment-api
    app_port: 8080

  tasks:
    - name: Render application config
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /opt/myapp/app.conf
        owner: root
        group: root
        mode: '0644'
```

Run:

```bash
ansible-playbook -i inventory.ini deploy.yml
```

Generated `/opt/myapp/app.conf`:

```
APP_NAME=payment-api
APP_PORT=8080
```

`copy` **vs** `template`

`copy`	`template`
Copies static file content	Renders dynamic content
Does not evaluate Jinja2 expressions in the source file	Evaluates Jinja2 expressions
Best for scripts, certificates, static files	Best for application and service configuration

Important: Both modules can preserve file permissions and ownership. The key difference is whether the source content needs rendering.

---

57. Dynamic Configuration Generation

Purpose: Use variables, conditions, and loops together to generate a complete configuration suited to a particular environment or server.

Real-world example: Generate an Nginx reverse-proxy configuration for different backend servers.

Step 1 — Define variables

```yaml
vars:
  environment: production
  nginx_port: 80
  backend_servers:
    - "10.0.1.10:3000"
    - "10.0.1.11:3000"
```

Step 2 — Create the template

`templates/nginx.conf.j2`

```nginx
upstream backend {
{% for server in backend_servers %}
    server {{ server }};
{% endfor %}
}

server {
    listen {{ nginx_port }};

    {% if environment == "production" %}
```

```
    access_log /var/log/nginx/access.log;
    {% endif %}

    location / {
        proxy_pass http://backend;
    }
}
```

Step 3 — Render the template

```yaml
- name: Generate Nginx configuration
  hosts: web
  become: true

  vars:
    environment: production
    nginx_port: 80
    backend_servers:
      - "10.0.1.10:3000"
      - "10.0.1.11:3000"

  tasks:
    - name: Render Nginx configuration
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/conf.d/myapp.conf
        owner: root
        group: root
        mode: '0644'
      notify: Validate and reload Nginx

  handlers:
    - name: Validate and reload Nginx
      ansible.builtin.command: nginx -t
      changed_when: false
      notify: Reload Nginx

    - name: Reload Nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Step 4 — Understand the workflow

Variables

Port, environment, backend servers

↓

Jinja2 template

Variables + conditions + loops

↓

Generated Nginx configuration

Actual server addresses and configuration

↓

Validate and reload

Apply the change only after validation succeeds

Important: Handlers run when notified by a task that reports a change. In this example, validation must succeed before the reload handler is notified. For production, consider using a dedicated validation task or a carefully structured handler chain, and test it against your target Ansible version.

Quick revision

Topic	Remember this
52. Jinja2	Templating engine used to generate text
53. Variables	`{{ variable }}` inserts a value
54. Conditions	`{% if condition %}` selects content
55. Loops	`{% for item in list %}` repeats content
56. `.j2` files	Store reusable templates
57. Dynamic generation	Combine variables, conditions, and loops

Interview essentials

1. **Why use templates?** To generate different configuration files from one reusable source.
2. **What does `{{ }}` do?** Evaluates an expression and inserts its result.
3. **What does `{% %}` do?** Controls template logic, such as loops and conditions.
4. **Can one template generate different files for different servers?** Yes, using host-specific or group-specific variables.

5. **What happens when a rendered file changes?** The template task reports a change, allowing handlers to restart or reload dependent services.
6. **What is the difference between `when` and `{% if %}`?** `when` controls task execution; `{% if %}` controls generated content.

Study priority: Practise variable substitution, one condition, one loop, and one complete Nginx template. These cover the core concepts without needing a large lab.